Support de Cours

Coder avec l'IA avec GitHub Copilot (cas réel Java/Angular)

Fabien ROHRER *Modifié le 23 février 2026*

L'IA en perspective - Fondations et état des lieux **2**
Définitions 2
Terminologie 3
Fonctionnement interne d'une LLM : probabilités, tokens, contexte 3
3.1 Ce qu'une LLM manipule : du texte → des unités (tokenization) 3
3.2 Token : définition opérationnelle 4
3.3 Contexte : la mémoire de travail (et sa limite) 4
Variabilité : pourquoi deux réponses peuvent différer 4
Pourquoi ça marche si bien pour le code 5
Ce que ces modèles font très bien (typiquement) 5
Ce qu'ils font moins bien 5
Applications au code : relier ça à GitHub Copilot (Java/Angular) 5
Limites : hallucinations, vérité, raisonnement 6
Évolution rapide : état des lieux + mise en garde 7
Conclusion : 3 takeaways 7
Panorama des usages concrets **8**
1) L'IA comme assistant du développeur (productivité individuelle) 8
2) L'IA pour comprendre, explorer et analyser le code 9
3) L'IA comme composant applicatif dans un produit 9
4) L'IA pour l'automatisation et l'aide à la décision 10
5) L'IA via API et MCP (structurer l'intégration) 10
Conclusion : repères rapides (3 takeaways) 11

L’IA en perspective - Fondations et état des lieux

Aujourd’hui, l’objectif de la journée est très concret : **apprendre à coder avec l’IA**, en particulier **GitHub Copilot**, dans un **cas réel Java/Angular** (dans IntelliJ), pour accélérer et améliorer des tâches de développement au quotidien.

Avant de passer à la pratique, on démarre par une mise en perspective courte : on veut **poser un socle commun** et éviter deux pièges fréquents :

- surestimer l’IA (“elle a toujours raison”),
- ou la sous-estimer (“ça ne sert qu’à générer du boilerplate”).

À la fin de ce chapitre d’introduction, vous aurez trois repères simples :

1. **Comprendre ce que fait réellement une IA de type LLM** (et ce qu’elle ne fait pas).
2. **Savoir pourquoi elle est souvent très efficace sur le code**, et dans quels cas elle peut se tromper.
3. **Avoir des règles d’usage** pour rester fiable et pro : qualité, sécurité, confidentialité.

Le message clé qui va guider tout le reste de la formation : **Copilot est un copilote**. Il propose, **vous décidez** (et vous validez).

Définitions

Qu’est-ce que l’IA (et ce qu’elle n’est pas)?

Le terme **IA** est un terme générique. Il regroupe des approches très différentes : systèmes à règles, statistiques, machine learning, assistants conversationnels, etc.

Dans cette formation, on ne parlera que d’**IA générative**, basée sur des **LLM (Large Language Models)** : des modèles capables de produire du texte et du code à partir d’un contexte. C’est ce qui alimente GitHub Copilot dans notre cas Java/Angular.

Pour éviter les malentendus, trois points essentiels :

- Ce n’est **pas une conscience** : il n’y a ni intention, ni compréhension humaine.
- Ce n’est **pas un moteur de vérité** : une réponse peut être fluide et fausse.
- Ce n’est **pas un remplaçant du développeur** : sans contexte métier clair, il suppose.

Pour bien utiliser l’IA, il faut comprendre **ce qu’elle optimise réellement** — et non ce qu’on projette sur elle: **la probabilité du prochain token compte tenu du contexte**.

Elle n’optimise pas la vérité, la performance, la sécurité, la conformité métier et la compilation.

Terminologie

Machine Learning (ML)
Le ML, c'est quand un système **apprend des régularités à partir de données**, plutôt que d'être programmé uniquement avec des règles explicites. En dev, ça se traduit par : "il a appris des patterns" et il peut **généraliser** à des cas proches.

Réseaux de neurones
C'est une famille de modèles de ML où on empile des "couches" de calcul. Pendant l'entraînement, le modèle ajuste des **poids** pour transformer une entrée en sortie de plus en plus efficacement. On peut le voir comme un gros système de fonctions paramétrées qui s'optimise.

Deep Learning
C'est simplement des réseaux de neurones **profonds** (beaucoup de couches), particulièrement performants pour le langage, l'image, l'audio. Les LLM sont une application majeure du deep learning.

LLM (Large Language Model)
Un LLM est un modèle de deep learning spécialisé dans le **langage**. Il apprend la structure du texte… et du code : style, conventions, patterns, API courantes. Dans notre contexte, c'est ce qui permet à Copilot de proposer des complétions ou des fonctions entières en se basant sur ce que tu écris et ce que le projet "ressemble".

"Thinking" / Raisonnement
Faire produire à l'IA une réponse en plusieurs passes (plan → tentative → critique → correction), en l'incitant à vérifier ses hypothèses, à repérer ses trous, et à réviser avant de livrer le résultat final.

- Planification : "voici les étapes que je vais suivre"
- Auto-critique : "où je peux me tromper / quelles hypothèses je fais"
- Révision : "je corrige / j'améliore / je sécurise"

Fonctionnement interne d'une LLM : probabilités, tokens, contexte

L'objectif ici n'est pas de faire un cours de data science, mais de construire un **modèle mental simple** : *comment une LLM "lit" une entrée, et comment elle "écrit" une sortie*. On introduit les concepts dans l'ordre où ils deviennent nécessaires.

3.1 Ce qu'une LLM manipule : du texte → des unités (tokenization)

Une LLM ne travaille pas directement avec des phrases "comme nous". Elle transforme d'abord tout ce qu'on lui donne (texte ou code) en une suite d'unités standardisées : c'est la **tokenization**.

- **Tokenization** : découpage d'un texte en éléments plus petits, adaptés au modèle.

- Ces unités s'appellent des **tokens**.

Pourquoi on commence par là : si on ne comprend pas ce que la machine "voit", on ne comprend pas ses limites.

3.2 Token : définition opérationnelle

Un **token** est une unité de lecture/écriture pour le modèle.
Ce n'est **pas** forcément un mot : ça peut être un mot entier, un bout de mot, un symbole, une ponctuation, parfois un morceau d'identifiant ou de nombre.

Ce que ça implique :

- Le modèle "pense" (au sens calcul) **token par token**.
- La longueur d'un input, d'une réponse, ou d'une "mémoire" se mesure en **tokens**, pas en lignes ni en pages.

3.3 Contexte : la mémoire de travail (et sa limite)

Une fois tokenisé, l'ensemble des tokens fournis au modèle constitue son **contexte**.

Le contexte, c'est :

- la consigne,
- le texte déjà échangé,
- et ce que le modèle a sous les yeux au moment de répondre.

Point crucial : ce contexte est **limité**. Il y a une taille maximale (une fenêtre) : tout ne rentre pas.

Conséquences pédagogiques (générales, sans outil) :

- Si une information importante n'est pas dans le contexte, le modèle ne peut pas s'y appuyer.
- Il va alors soit rester vague, soit **compenser** avec des hypothèses plausibles.

Génération : prédire la suite, pas "chercher la vérité"

Une fois le contexte en place, la génération suit une logique simple :

1. le modèle lit le contexte (tokens)
2. il calcule des probabilités : "quel token pourrait venir ensuite ?"
3. il choisit un token
4. il recommence jusqu'à former une réponse

C'est le modèle mental central : **une LLM est un moteur de continuation**.
Elle optimise la **plausibilité** de la suite, pas la vérité, ni la correction.

Variabilité : pourquoi deux réponses peuvent différer

La sélection des tokens peut être :

- plus **stable** (choix plus conservateur),
- ou plus **variée** (choix plus exploratoire).

Donc, à contexte identique, on peut obtenir des réponses différentes — pas parce que le modèle “change d’avis”, mais parce qu’il y a plusieurs suites plausibles.

Pourquoi ça marche si bien pour le code

Le code est un **langage très contraint**. Il a une syntaxe stricte, des règles de format (lint, style), et surtout énormément de régularités. Dans la plupart des projets, on retrouve les mêmes structures : accès aux données, Data Transfer Object (DTO), validations, transformations, tests, patterns de framework. Résultat : un modèle entraîné sur de grandes quantités de code apprend des **motifs réutilisables** et peut proposer rapidement une suite “probable” qui ressemble à ce que ferait un développeur dans un cas standard.

Ce que ces modèles font très bien (typiquement)

Ils sont particulièrement utiles quand la tâche est **prévisible** ou **déjà illustrée** dans le code existant :

- compléter du code répétitif (boilerplate, mapping DTO, validations, wiring) ;
- générer une fonction “dans le même style” à partir d’un exemple déjà présent ;
- proposer des tests unitaires “standards” (structure, cas nominaux) ;
- aider à reformuler ou refactoriser localement (renommage, extraction de méthode, simplification) ;
- expliquer ce que fait un fichier ou un morceau de code, ce qui est pratique pour l’onboarding ou le legacy.

Ce qu’ils font moins bien

À l’inverse, les difficultés apparaissent dès qu’il faut **comprendre des intentions** non écrites ou raisonner “système” :

- règles métier implicites (ce qui n’est pas explicite dans le code ou les tests) ;
- architecture globale et cohérence long-terme (choix de design, dépendances, conventions spécifiques) ;
- cas limites non décrits (pas de tests, pas de specs, pas d’exemples) ;
- diagnostic de bug complexe, surtout quand le symptôme est loin de la cause.

Applications au code : relier ça à GitHub Copilot (Java/Angular)

À ce stade, on peut relier le modèle mental “tokens → contexte → génération” à l’usage dev : Copilot est utile dès qu’on sait **quoi lui demander** et **comment cadrer**.

On peut résumer Copilot en trois modes de travail complémentaires :

- **Complétion inline** : accélérer l'écriture locale (la "suite la plus probable" dans un fichier).

- **Chat** : formuler un objectif, demander une explication, proposer une approche, générer une variante, demander des tests.

- **Édition multi-fichiers (selon environnement)** : demander une modification structurée et appliquer des changements à plusieurs endroits depuis une consigne unique.

Sur un cas Java/Angular, les usages "rentables" reviennent souvent :

- générer un squelette (DTO, service, contrôleur, composant) à partir d'un exemple existant ;

- produire des variantes (même logique, autre endpoint, autre écran) ;

- proposer des tests unitaires "nominaux" pour verrouiller un comportement ;

- refactor local (extraction, renommage, simplification) ;

- expliquer un fichier legacy pour accélérer l'onboarding.

Point de méthode : plus la demande est **contrainte** (format attendu, conventions, exemples, règles), plus la sortie est exploitable.

Limites : hallucinations, vérité, raisonnement

Une IA générative peut produire une réponse très convaincante… et incorrecte. La limite centrale, c'est qu'elle optimise la **plausibilité**, pas la vérité.

Hallucinations (en dev : erreurs plausibles)
Ça peut prendre la forme d'une API inventée, d'un paramètre erroné, d'une hypothèse implicite fausse, ou d'un "bon raisonnement" basé sur une mauvaise prémisse. GitHub insiste sur le besoin d'utiliser Copilot de façon responsable en gardant en tête ses limites.

Faux positifs (notamment en revue)
Même quand Copilot "review" ou commente, il peut signaler des problèmes qui n'existent pas (malentendu du code, contexte incomplet). Ces retours doivent être lus comme des suggestions, pas comme un verdict.

Règle simple de mitigation

- Traiter la sortie comme une **proposition**.
- Valider systématiquement : compilation, tests, exécution, revue.

- Réduire le risque en travaillant par petites étapes (itérations courtes) et en rendant explicites les hypothèses/contraintes.

Évolution rapide : état des lieux + mise en garde

Le paysage évolue vite : les modèles, les intégrations IDE, et les fonctionnalités disponibles changent régulièrement. Par exemple, GitHub documente des capacités comme les changements multi-fichiers et publie des améliorations côté JetBrains au fil des mises à jour.

La conséquence pédagogique : on ne base pas la formation sur “une feature exacte à une date”, mais sur des invariants :

- **contexte** : ce que l’outil a réellement sous les yeux ;
- **contrainte** : comment on formule une demande testable et alignée au projet ;
- **validation** : comment on prouve qu’une suggestion est correcte.

Enfin, à l’échelle d’une équipe, il faut aussi intégrer une dimension gouvernance : fonctionnalités réellement disponibles dans l’IDE, paramètres de l’organisation, et règles d’usage (données sensibles). La “feature matrix” aide à clarifier ce qui est supporté selon l’IDE/versions.

Conclusion : 3 takeaways

Une IA générative produit du **plausible** à partir d’un contexte ; ce n’est pas un juge de vérité.
La qualité dépend surtout de **ce que tu fournis** : contexte pertinent + contraintes claires + itérations courtes.
Le développeur reste responsable : **tests, compilation, review, sécurité** — Copilot accélère, il ne garantit pas.

Panorama des usages concrets

Aujourd'hui, on ne cherche pas à "tout faire avec l'IA". L'objectif est plus simple : **avoir une carte mentale** des usages réellement utiles en développement, et savoir **où l'IA apporte du gain** (et où elle apporte surtout du bruit).

On peut regrouper les usages en **4 familles** :

1. **L'IA comme assistant du développeur** (productivité individuelle)
2. **L'IA pour comprendre / explorer / analyser le code**
3. **L'IA comme composant applicatif dans un produit**
4. **L'IA pour l'automatisation et l'aide à la décision**

Le fil rouge reste le même : **l'IA propose, vous décidez (et vous validez).**

1) L'IA comme assistant du développeur (productivité individuelle)

Ici, l'IA est surtout utile quand la tâche est **répétitive**, **spécifiable**, et qu'on peut **valider rapidement** (compilation, tests, exécution).

Génération de code (boilerplate "intelligent")

- Générer une fonction à partir d'un objectif clair
- Produire un squelette (service, contrôleur, DTO, composant Angular)
- Proposer un exemple standard (mapping, validation, pagination, etc.)

Assistance au refactoring (améliorer un code existant)

- Simplifier une logique
- Renommer et clarifier des intentions
- Extraire des méthodes, réduire la complexité locale
- Améliorer la lisibilité (sans changer le comportement)

Génération de tests (verrouiller rapidement)

- Squelette de tests unitaires (JUnit/Jest)
- Cas nominaux + premières variations
- Idées de cas limites (à valider par vous)

Débogage assisté (accélérer la boucle de feedback)

- Expliquer une erreur / stacktrace
- Proposer des hypothèses plausibles
- Suggérer des logs ou points de contrôle

- Aider à isoler une cause probable

Documentation et complétion

- Javadoc / README / doc technique de module
- Exemples d'utilisation (API, endpoints, composants)
- Commentaires utiles (sans sur-documenter)

> Point de méthode : plus votre demande contient **contraintes + exemples + format attendu**, plus la sortie devient exploitable.

2) L'IA pour comprendre, explorer et analyser le code

Quand on arrive sur un projet ou un module legacy, l'IA peut jouer le rôle de **lecteur rapide**. Elle ne "comprend" pas comme un humain, mais elle sait **résumer** et **pointer des éléments structurants**.

Compréhension de bases de code existantes

- "Que fait ce module ? quelles responsabilités ?"
- "Quels sont les points d'entrée ?"
- "Quels sont les flux principaux ?"

Navigation dans des projets complexes

- "Où se trouve la logique X ?"
- "Quelles classes/fichiers sont impliqués ?"
- "Quelles dépendances je risque d'impacter ?"

Résumés et cartographie

- Résumé de fonctionnalités
- Identification des composants "cœur"
- Mise en évidence des conventions locales (style, patterns)

Analyse statique assistée (à lire comme des suggestions)

- Qualité / complexité / smells
- Conventions, style, répétitions
- Risques potentiels (null, erreurs, exceptions)

> Mise en garde : si l'information n'est pas dans le contexte, l'IA **comble** avec des hypothèses plausibles. On valide toujours sur le code réel et les tests.

3) L'IA comme composant applicatif dans un produit

Ici, l'IA n'est plus "dans l'IDE" : elle devient une **fonctionnalité** visible côté utilisateur. Le sujet central n'est pas seulement technique : c'est aussi **UX, fiabilité, contrôle**.

Fonctionnalités exposées aux utilisateurs finaux

- Recherche intelligente / sémantique (dans une base documentaire, FAQ, tickets)
- Assistants conversationnels intégrés (support, onboarding)
- Aide contextuelle dynamique (selon l'écran et l'action)
- Génération de contenu (textes, résumés, suggestions)

IA intégrée comme service / brique fonctionnelle

- Un endpoint "résume / classe / extrait / reformule"
- Une brique de tri / détection / normalisation
- Une aide à la rédaction (avec garde-fous)

> Repère simple : dès que c'est exposé à un utilisateur final, il faut penser **erreurs**, **ambiguïtés**, **explicabilité**, et **fallback**.

4) L'IA pour l'automatisation et l'aide à la décision

L'IA est très efficace pour traiter du volume : **trier**, **prioriser**, **détecter des motifs**. Mais elle ne remplace pas la décision : elle **prépare** et **suggère**.

Cas typiques

- Analyse automatique de données métier (résumés, extraction, regroupements)
- Détection d'anomalies ou tendances (signaux faibles)
- Priorisation / tri / classification (tickets, incidents, demandes)
- Support à la décision humaine (options + critères + risques)

> Repère simple : plus l'enjeu est élevé, plus on garde un **humain dans la boucle**.

5) L'IA via API et MCP (structurer l'intégration)

On peut utiliser l'IA comme un service programmable :

- via **API** (appel modèle, envoi contexte, récupération résultat),
- via des **outils/connecteurs** (accès à des sources : repo, docs, base, services).

Ce qui fait la différence en pratique

- **Contexte** : ce que vous fournissez (données, extraits, règles, exemples)
- **Contrôle** : format attendu (liste, tableau, JSON), contraintes, critères
- **Validation** : tests, compilation, logs, revue
- **Coût/latence** : taille du contexte, nombre d'appels, stratégie d'itération

MCP (Model Context Protocol) — idée en une phrase

- Structurer l'accès du modèle à du contexte et à des outils de façon standardisée, pour en faire une **brique fonctionnelle** (analyse, génération structurée, assistance contextuelle, automatisation).

Enjeux clés (à garder en tête)

- Gestion du contexte (ce que le modèle voit vraiment)
- Sécurité des données (confidentialité, secrets)
- Contrôle des résultats (formats, règles, vérification)
- Coûts (tokens, nombre d'itérations)

Conclusion : repères rapides (3 takeaways)

1. Les meilleurs usages sont ceux qui sont **spécifiables** et **vérifiables** rapidement (tests, compilation, exécution).
2. L'IA est excellente pour **accélérer** (draft), mais la fiabilité vient de votre **validation** (review, tests).
3. En cas de doute : **réduire la taille du problème**, travailler par itérations courtes, et rendre explicites les contraintes.